

EXPERIMENT 1

NUMERICAL SOLUTION OF ORDINARY DIFFERENTIAL EQUATIONS

Objectives

- a) To write Scilab programs to solve the following ordinary differential equations (ODEs) using the in-built ode function:
- i. $\frac{dy}{dx} = e^x$, with $y(0) = 0$.
 - ii. $\frac{dy}{dx} + e^{-x}y = x^2$, with $y(0) = 0$.
 - iii. $\frac{d^2y}{dx^2} + 2\frac{dy}{dx} = -y$, with $y(0) = 1$ and $\frac{dy}{dx}(0) = 0$.
 - iv. $\frac{d^2y}{dx^2} + e^{-x}\frac{dy}{dx} = -y$, with $y(0) = 1$ and $\frac{dy}{dx}(0) = 0$.
- b) To visualize the solutions over the domain $0 \leq x \leq 5$ using plotting functions in Scilab.

Theory

Many physical systems exhibit behaviour that can be represented by nth-order differential equations. While simple equations may have analytical solutions, complex systems often require numerical software for evaluation. Scilab provides a powerful in-built command, ode, which uses efficient backend algorithms (such as Runge-Kutta fourth-order) to solve initial value problems.

In general, a first order ODE is written as

$$\frac{dy}{dx} = f(x, y)$$

with initial conditions $y(x_0) = y_0$.

A second order ODE is written as

$$\frac{d^2y}{dx^2} = g(x, y)\frac{dy}{dx} + f(x, y)$$

with initial conditions $y(x_0) = y_0$ and $\frac{dy}{dx} = v_0$.

Procedure

To solve Ordinary Differential Equations (ODEs) in Scilab, the primary tool used is the in-built ode command. This command solves initial value problems numerically using efficient back-end algorithms.

First order ODEs

A first-order ODE must be expressed in the standard form:

$$\frac{dy}{dx} = f(x, y)$$

- I. Write a Scilab function representing $f(x, y)$. The function should have two input arguments: the independent variable (e.g. x) and the dependent variable (e.g. y).
- II. Define the starting point (x_0) and the value of the solution at that point (y_0).
- III. Create a vector (using `linspace` or the `:` operator) for the values of x where the solution is needed.
- IV. Call the `ode` command using the syntax: `y_sol = ode(y0, x0, x, f)`.
- V. Use the `plot(x, y_sol(1, :))` command to visualize the results.

Second order ODEs

Scilab does not have a separate command for second-order ODEs; instead, a “trick” is used to reduce the second-order equation into a system of two coupled first-order equations solved simultaneously.

- I. Transform the second-order equation into a system.
 - a) Define two variables:

$$\begin{aligned} y_1 &= y \\ \Rightarrow y_2 &= \frac{dy}{dx} \end{aligned}$$

- b) Express their derivatives:

$$\begin{aligned} \frac{dy_1}{dx} &= y_2 \\ \frac{dy_2}{dx} &= \text{original equation re-arranged} \end{aligned}$$

- II. Create a function where the input y is a vector $[y_1; y_2]$ and the output dy is a column vector containing their derivatives.
- III. Provide a column vector containing the initial value of y and the initial value of its first derivative, e.g. `y0 = [initial_y; initial_dydx]`.
- IV. Call the `ode` command as before: `y_sol = ode(y0, x0, x, f_coupled)`.
- V. The resulting `y_sol` will be a matrix. The first row represents the solution $y(x)$, and the second row represents the derivative dy/dx .
- VI. Use the `plot(x, y_sol(1, :))` command to visualize the results.

Scilab Code

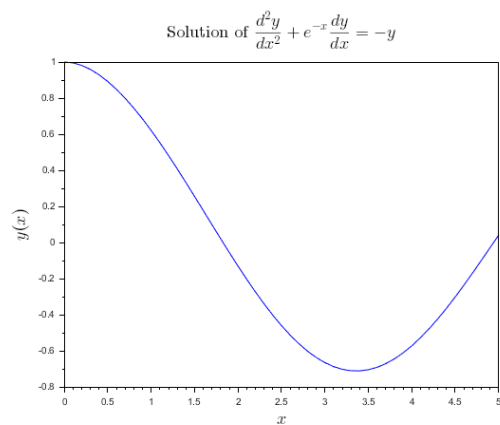
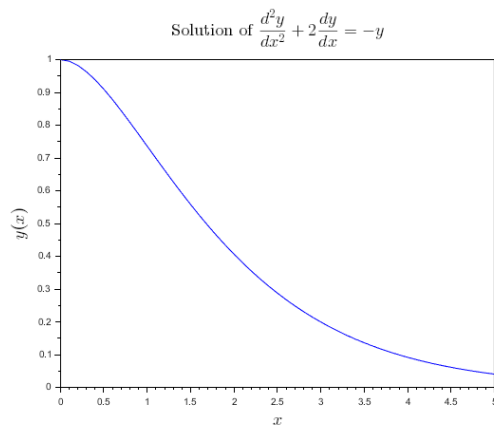
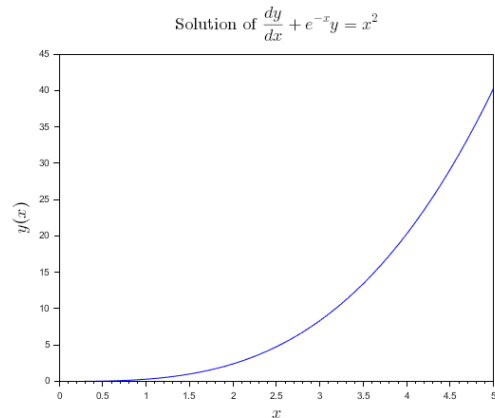
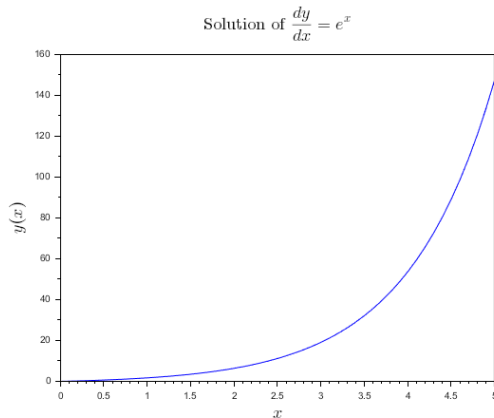
```

1 function dy = f1(x, y)
2     dy = exp(x);
3 endfunction
4
5 function dy = f2(x, y)
6     dy = x^2 - exp(-x)*y;
7 endfunction
8
9 function dy = f3(x, y)
10    dy(1) = y(2);
11    dy(2) = -y(1) - 2*y(2);
12 endfunction
13
14 function dy = f4(x, y)
15    dy(1) = y(2);
16    dy(2) = -y(1) - exp(-x)*y(2);
17 endfunction
18
19 x = 0:0.1:5;
20 x0 = 0; y0 = 0;
21 y_sol1 = ode(y0, x0, x, f1);
22 y_sol2 = ode(y0, x0, x, f2);
23 y0_coupled = [1; 0];
24 y_sol3 = ode(y0_coupled, x0, x, f3);
25 y_sol4 = ode(y0_coupled, x0, x, f4);
26
27 scf(1); plot(x, y_sol1(1, :));
28 title("$\text{Solution of } \frac{dy}{dx}=e^x$", "fontsize", 4);
29 xlabel("$x$", "fontsize", 4); ylabel("$y(x)$", "fontsize", 4);
30
31 scf(2); plot(x, y_sol2(1, :));
32 title("$\text{Solution of } \frac{dy}{dx}+e^{-x}y=x^2$", "
    fontsize", 4);
33 xlabel("$x$", "fontsize", 4); ylabel("$y(x)$", "fontsize", 4);
34
35 scf(3); plot(x, y_sol3(1, :));
36 title("$\text{Solution of } \frac{d^2y}{dx^2}+2\frac{dy}{dx}=-y$", "
    ", "fontsize", 4);
37 xlabel("$x$", "fontsize", 4); ylabel("$y(x)$", "fontsize", 4);
38
39 scf(4); plot(x, y_sol4(1, :));
40 title("$\text{Solution of } \frac{d^2y}{dx^2}+e^{-x}\frac{dy}{dx}=-y$", "
    ", "fontsize", 4);
41 xlabel("$x$", "fontsize", 4); ylabel("$y(x)$", "fontsize", 4);

```

Output

Plots of the solutions of given ODEs:



Results

- Scilab scripts were successfully developed to solve the four given first and second-order differential equations using the in-built ode function.
- The solutions were visualized through plots, confirming the expected physical behaviour of the systems.